

Designing for Non- Functional Properties

1. Efficiency

Efficiency is a non-functional property that refers to the ability of a system to perform its operations using minimum resources such as time, memory, CPU, and network bandwidth while delivering maximum output. An efficient system ensures fast response time, quick processing, and optimal resource utilization. It focuses on reducing delays, avoiding unnecessary computations, and improving overall system performance even under heavy workloads.

a) Software Components and Efficiency

- Efficiency at component level depends on how well each module utilizes CPU, memory, and processing time.
- Components should use optimized algorithms (e.g., $O(n \log n)$ instead of $O(n^2)$) to reduce execution time.
- Memory management is important -avoid memory leaks, unnecessary object creation, and large data storage.
- Reusability of components improves efficiency by reducing redevelopment time and resource usage.
- Components should avoid redundant operations and support lazy loading (load only when needed).

b) Software Connectors and Efficiency

- Connectors represent communication (function calls, APIs, message passing), so inefficient connectors can slow down the entire system.
- Frequent communication between components increases overhead → minimize unnecessary interactions.
- Use lightweight protocols (REST, gRPC) instead of heavy ones where performance matters.
- Prefer asynchronous communication to avoid blocking system resources.
- Data transfer should be optimized using compression and minimal payload formats like JSON.

c) Architectural Configuration and Efficiency

- Architecture defines overall performance; poor design leads to bottlenecks.
- Use caching mechanisms (browser cache, server cache, database cache) to reduce repeated computation.
- Introduce load balancing to distribute workload across multiple servers.
- Use parallel processing and concurrency for faster execution.
- Avoid single points of failure and performance bottlenecks in system design.

2. Complexity

Complexity is the measure of how difficult a software system is to understand, design, develop, and maintain. It arises due to large system size, multiple components, and intricate interactions between them. High complexity makes systems error-prone and hard to modify, while low complexity improves readability and maintainability. Managing complexity involves modularization, abstraction, and proper system design.

a) Software Components and Complexity

- Complexity increases when components are large and perform multiple tasks.
- Apply Single Responsibility Principle (SRP) so each component does only one function.
- High cohesion within components reduces confusion and improves maintainability.
- Use abstraction to hide internal logic and expose only necessary details.
- Avoid deep nesting, long functions, and unclear logic to reduce cognitive load.

b) Software Connectors and Complexity

- Complex interactions between components make the system hard to understand and debug.
- Use **well-defined APIs** with clear input/output specifications.
- Avoid tight coupling (where one component depends heavily on another).
- Prefer **message queues or event-driven communication** to simplify dependencies.
- Document all interfaces and communication clearly.

c) Architectural Configuration and Complexity

- A complex architecture (too many layers, services) becomes difficult to manage.
- Use layered architecture, modular architecture, or micro-services to structure the system.
- Apply design patterns like MVC, client-server, etc., to manage complexity.
- Keep architecture simple and scalable rather than over-engineered.
- Maintain proper diagrams and documentation for clarity.

3. Scalability

Scalability is the ability of a system to handle increasing amounts of work, users, or data without affecting its performance or efficiency. A scalable system can grow by adding resources such as servers, memory, or processing power. It ensures that the system continues to perform well during peak loads and supports future expansion through horizontal or vertical scaling.

a) Software Components and Scalability

- Components should be designed to handle increasing load without modification.
- Stateless components are preferred because they can be easily replicated.
- Avoid hard coding limits (like fixed number of users or data size).
- Components should support **load distribution** and parallel execution.
- Use efficient data handling to support large-scale operations.

b) Software Connectors and Scalability

- Communication should not become a bottleneck as users increase.
- Use **asynchronous messaging systems (Kafka, RabbitMQ)** to handle high traffic.
- Implement **load balancing** to distribute requests evenly.
- Avoid synchronous blocking calls in high-load systems.
- Ensure connectors can handle high throughput and concurrent requests.

c) Architectural Configuration and Scalability

- Use **distributed architecture** to scale horizontally (adding more servers).
- Cloud-based systems (AWS, Azure) support dynamic scaling.
- Micro-services architecture allows scaling individual services independently.
- Use database sharding (Sharding means splitting one large database into smaller parts (shards).) and replication for large data handling.
- Design system to handle peak loads efficiently.

4. Heterogeneity

Heterogeneity refers to the capability of a system to operate in an environment that includes different hardware platforms, operating systems, programming languages, and network technologies. A heterogeneous system ensures smooth communication and integration among diverse components by using standard protocols and interfaces, making it flexible and widely compatible.

a) Software Components and Heterogeneity

- Components should work across different platforms (Windows, Linux, mobile).
- Use platform-independent languages like **Java, Python, web technologies**.
- Ensure compatibility with different hardware and environments.
- Design reusable components that can be deployed in multiple systems.

b) Software Connectors and Heterogeneity

- Different systems communicate using standard protocols like **HTTP, REST, SOAP**.
- Use **common data formats (JSON, XML)** for interoperability.
- Middleware acts as a bridge between different technologies.
- Handle differences in network speed, formats, and protocols effectively.

c) Architectural Configuration and Heterogeneity

- Architecture should support integration of diverse systems.
- Use **Service-Oriented Architecture (SOA)** or microservices.
- APIs help connect systems built using different technologies.
- Ensure system works across web, mobile, and desktop platforms.
- Design for interoperability and flexibility.

5. Adaptability

Adaptability is the ability of a system to **adjust to changes in user requirements, environment, or technology with minimal effort and cost**. An adaptable system can be easily modified, extended, or upgraded without affecting its core functionality. It supports flexibility and future growth by allowing **easy integration of new features or components**.

a) Software Components and Adaptability

- Components should be designed to easily accommodate changes.
- Avoid hardcoding values; use configuration files and environment variables.

- Components should support extension without modifying existing code (Open-Closed Principle).
- Design for future requirements and upgrades.

b) Software Connectors and Adaptability

- Use loosely coupled connectors so changes in one component do not affect others.
- APIs should be versioned to support updates without breaking systems.
- Allow dynamic integration of new services or modules.
- Support flexible communication mechanisms.

c) Architectural Configuration and Adaptability

- Use modular and layered architecture for easy updates.
- Allow addition/removal of components without affecting the whole system.
- Use plug-in architecture to extend functionality.
- Adopt agile architecture practices for continuous improvement.

6. Dependability

Dependability is the ability of a system to be reliable, available, safe, and secure, ensuring that users can trust it to function correctly over time. It includes aspects such as fault tolerance, error handling, data security, and system availability. A dependable system continues to operate correctly even in the presence of failures and protects data from unauthorized access.

a) Software Components and Dependability

- Components must be reliable and produce correct outputs consistently.
- Include proper error handling, exception management, and validation checks.
- Implement security features like authentication and authorization.
- Components should recover from failures without crashing the system.

b) Software Connectors and Dependability

- Communication must be reliable and secure.
- Use **encryption (HTTPS)** to protect data.
- Implement retry mechanisms, acknowledgments, and timeouts.
- Ensure data consistency and integrity during transmission.

c) Architectural Configuration and Dependability

- Use redundancy (backup servers, failover systems) for reliability.
- Ensure high availability with minimal downtime.
- Implement disaster recovery mechanisms.
- Continuous monitoring and testing improve system trustworthiness.
- Security architecture protects against threats and attacks.